Programare Python Curs 3 (2023 - 2024)

Tipuri de date singulare

Siruri de caractere (continuare)

Unicode

Unicode este un standard internațional pentru reprezentarea caracterelor în format numeric, astfel încât orice simbol scris - litere, cifre, semne de punctuație, caractere speciale sau emoji are un code point - un numar intreg, de obicei exprimat in hexazecimal, care poate avea valori intre 0 pana la 1.114.111 (10FFFF).

Intervale

- U+0000 - U+007F → ASCII (basic Latin)
- U+0080 - U+FFFF → BMP (Basic Multilingual Plane, common scripts)
- U+10000 - U+10FFFF → supplementary (emoji, rare scripts)

Pentru a reprezenta toate code-pointurile din unicode sunt necesari maximum 21 de biti.

Reprezentarea in memorie a caracterelor unicode trebuie ales un pattern de biti, numit encoding. Din multe motive, exista mai multe moduri de reprezentare a code-pointurilor - utf8, utf16, utf32. Fiecare cu problemele sale.

- utf32 stocheaza fiecare caracter pe 32 de biti. Acest format permite toate code-pointurile dar este extrem de ineficient - majoritatea textului de pe un sistem informatic obisnuit este compus din caractere ASCII. Codul escape pentru acesta este \U
- utf8 stocheaza fiecare caracter intr-un numar variabil de biti. Caracterele ASCII de baza sunt stocate pe un singur octet (char), caracterele extinse (diacritice) sunt stocate de obicei in 2 octeti, emoji-urile si alte caractere in grafii mai neobisnuite in mai multi octeti (pana la 4). Nu exista secventa escape pentru utf8
- utf16 aparut initial ca encoding de lungime fixa, cand se credea ca 65536 code-points sunt suficiente pentru reprezentarea tuturor caracterelor existente. Odata cu introducerea caracterelor asiatice nu a mai fost suficient dar utf16 fusese deja adoptat in windows si java asa ca a fost si el convertit in encoding variabil. Secventa escape este \u dar nu poate prelucra decat spatiul de baza al utf16 (unul dintre cele 65536 caractere de baza). \u nu suporta caracterele unicode apartinand code-pointurilor superioare.

Exista online baza de date cu aceste caractere, care afiseaza toate modurile de identificare a acestor caractere, denumire, variate tipuri de encoding, etc. https://unicode-table.com/en/24C3/

- |\N{Name} | Numele caracterului Unicode. Poate reprezenta orice caracter unicode, atata timp cat are un nume. |
- |\uxxxxxxxx | Character unicode reprezentat pe 16 biti, poate reprezenta code-point-urile aflate in intervalul U+0000 to U+FFFF (Basic Multilingual plane) |
- |\Uxxxxxxxx | Caracter unicode pe 32 de biti - oricare |

- |\000 | Character cu valoarea in octal - maximum 3 cifre, poate reprezenta doar caractere ASCII + ASCII extins |
- |\xhh | Character cu valoarea hex - doua cifre |

Pentru reprezentarea oricarui caracter unicode pe baza de code-point, se foloseste functia chr()

```
print(chr(0xFe96))
print(chr(0xD796))
print(chr(0xA841))
```

```
ت
휖
𡁁
```

Formatare a sirurilor

String formatting operator

Cea mai vechie metoda de formatare a șirurilor, similara cu printf. Se mai numeste si "modulo string formatting".

print(%)

Majoritatea specificatorilor din c sunt pastrati desi in multe situatii semnificatia lor este putin modificata, datorita diferentei tipurilor de date

Specificator	Descriere
d,i,u	Numar intreg in reprezentare zecimala
x,X	Numar intreg in reprezentare hexazecimala
o	Numar intreg in reprezentare octala
f,F	Float
e,E	Flotant in reprezentare stiintifica
g,G	Flotant in reprezentare aleasa automat in functie de dimensiune
c	Caracter
s,r,a	Sir de caractere

```
variabiluta = "%d %s costa %.2f lei" % (6, "mere", 10)

print(variabiluta)

6 mere costa 10.00 lei
```

%G alege cea mai potrivita reprezentare pentru numere de tip float dar daca trimitem un numar intreg se va face o conversie de tip, numarul devine float dar in lipsa zecimalelor cea mai potrivita reprezentare este fara acestea, numarul la print "pare" intreg dar nu este.

```
print("%G" % 120)
print("%G" % 1200)
print("%G" % 12000)
print("%G" % 120000)
print("%G" % 1200000)
print("%G" % 12000000)
print("%G" % 120000000)
```

```
120
1200
12000
120000
1.2E+06
1.2E+07
1.2E+08
```

Putem solicita un numar foarte mare de zecimale

```
print("%8.3f %4.111f %.2f    " % (12.234,12.234,12.234))
```

```
  12.234
12.233999999999999985789145284797996282577514648437500000000000000000000
000000000000000000000000000000000000000000000000 12.23
```

Ocazie cu care putem vedea cum arata numerele care sunt dificil de stocat in float

```
print("%4.111f" % 0.7)
print("%4.111f" % 0.3)
print("%4.111f" % (1/3))
```

```
0.699999999999999995559107901499373838305473327636718750000000000000000000
000000000000000000000000000000000000000000000000
0.299999999999999988897769753748434595763683319091796875000000000000000000
000000000000000000000000000000000000000000000000
0.333333333333333314829616256247390992939472198486328125000000000000000000
000000000000000000000000000000000000000000000000
```

metoda format a obiectului "sir de caractere"

```
nume1 = 'Ionescu'
nume2 = 'Popescu'
fnume1 = 'Alina'
fnume2 = 'Vasile'

strf = 'Ordinea va fi urmatoarea: mai intai {} {} si apoi {} {}'

print(strf)
print(strf.format(fnume1, fnume2, nume1, nume2))
```

```
Ordinea va fi urmatoarea: mai intai {} {} si apoi {} {}
Ordinea va fi urmatoarea: mai intai Alina Vasile si apoi Ionescu
Popescu

strfr = '{1} poate tranti o {0} {2}'

print(strfr.format('usa', 'Chuck Norris','rotativa'))

Chuck Norris poate tranti o usa rotativa

strfr = '{cc} poate tranti o {uu} {rr}'

print(strfr.format(uu = 'usa', cc='Chuck Norris', rr = 'rotativa'))

Chuck Norris poate tranti o usa rotativa
```

Format poate fi utilizat și pentru numere flotante, caz în care stringul este nu doar înlocuit

Syntax: {[index]:[width][.precision][type]}

The type can be used with format codes:

```
‘d’ for integers
‘f’ for floating-point numbers
‘b’ for binary numbers
‘o’ for octal numbers
‘x’ for octal hexadecimal numbers
‘s’ for string
‘e’ for floating-point in an exponent format

fln = 105.1234223

print('Numarul este {dist:1.5f}'.format(dist = fln))
print('Numarul este {dist:10.2f}'.format(dist = fln))

Numarul este 105.12342
Numarul este     105.12

print('Numarul este {dist:10.2f}'.format(dist = fln +2))

Numarul este     107.12
```

f-strings

F-strings utilizeaza direct numele variabilelor

```
nume1 = 'Ionescu'
nume2 = 'Popescu'
fnume1 = 'Alina'
fnume2 = 'Vasile'
```

```
strf = 'Ordinea va fi urmatoarea: mai intai {} {} si apoi {} {}'
fstringstrf = f"Ordinea va fi urmatoarea: mai intai {fnume1} {nume1}
si apoi {fnume2} {nume2}"

print(fstringstrf)
```

```
Ordinea va fi urmatoarea: mai intai Alina Ionescu si apoi Vasile
Popescu
```

F-strings permite alinierea

```
name = "Alice"
age = 35
height = 168

form_str = f"Name: {name:>10}, Age {age:5}, Height {height:<5.2f}"
print(form_str)
```

```
Name:      Alice, Age    35, Height 168.00
```

```
form_str = f"Name: ->{name:>10}<-, Age ->{age:5}<-, Height -
>{height:<25.2f}<-"
print(form_str)
```

```
Name: ->     Alice<-, Age ->   35<-, Height ->168.00
<-
```

```
name = "Alice"
age = 35
height = 168

form_str = f"Name: ->{name:>20}<-, Age ->{age:20}<-, Height -
>{height:<18.2f}<-"
print(form_str)

name = "Consuela"
age = 31
height = 165
form_str = f"Name: ->{name:>20}<-, Age ->{age:20}<-, Height -
>{height:<18.2f}<-"
print(form_str)

name = "Patricia"
age = 29
height = 172
form_str = f"Name: ->{name:>20}<-, Age ->{age:20}<-, Height -
>{height:<18.2f}<-"
print(form_str)

name = "Ion"
age = 38
```

```
height = 198
form_str = f"Name: ->{name:>20}<-, Age ->{age:20}<-, Height -
>{height:<18.2f}<-"
print(form_str)
```

```
Name: ->               Alice<-, Age ->                  35<-, Height -
>168.00            <-
Name: ->            Consuela<-, Age ->                  31<-, Height -
>165.00            <-
Name: ->            Patricia<-, Age ->                  29<-, Height -
>172.00            <-
Name: ->                 Ion<-, Age ->                  38<-, Height -
>198.00            <-
```

Operatii cu data si timp

Cel mai utilizat modul pentru operatii du date/timp in Python este datetime

```
from datetime import datetime, date, time

year = 1420
month = 1
day = 20
hour = 7
minute =  28
second = 15

dt = datetime(year, month, day, hour, minute, second)

print(dt)
```

```
1420-01-20 07:28:15
```

```
now = datetime.now()

print(now)
```

```
2026-04-03 21:08:33.813802
```

```
print(now.timestamp())
```

```
1775250513.813802
```

```
print(type(dt))
print(dir(dt))
```

```
<class 'datetime.datetime'>
['__add__', '__class__', '__delattr__', '__dir__', '__doc__',
```

```
'__eq__', '__format__', '__ge__', '__getattribute__', '__getstate__',
'__gt__', '__hash__', '__init__', '__init_subclass__', '__le__',
'__lt__', '__ne__', '__new__', '__radd__', '__reduce__',
'__reduce_ex__', '__repr__', '__rsub__', '__setattr__', '__sizeof__',
'__str__', '__sub__', '__subclasshook__', 'astimezone', 'combine',
'ctime', 'date', 'day', 'dst', 'fold', 'fromisocalendar',
'fromisoformat', 'fromordinal', 'fromtimestamp', 'hour',
'isocalendar', 'isoformat', 'isoweekday', 'max', 'microsecond', 'min',
'minute', 'month', 'now', 'replace', 'resolution', 'second',
'strftime', 'strptime', 'time', 'timestamp', 'timetuple', 'timetz',
'today', 'toordinal', 'tzinfo', 'tzname', 'utcfromtimestamp',
'utcnow', 'utcoffset', 'utctimetuple', 'weekday', 'year']
```

```
print(dt.hour)
print(dt.min)
print(dt.minute)
```

```
7
0001-01-01 00:00:00
28
```

Elementele de data / ora sunt formatate in foarte multe moduri, existand extrem de multe standarde. In majoritatea limbajelor exista doua functii:

strftime: produce un sir pe baza unui obiect de tip datetime cu un format definit

strptime: produce un obiect de tip datetime convertind un sir existent cu un format definit

Aceste doua metode/functii exista si in alte limbaje de programare si sunt inspirate din metodele cu acelasi nume din C standard library

```
print(now.strftime('%d-%m-%Y %H:%M'))
```

```
03-04-2026 21:08
```

Directive	Meaning	Example
%a	Abbreviated weekday name.	Sun, Mon, ...
%A	Full weekday name.	Sunday, Monday, ...
%w	Weekday as a decimal number.	0, 1, ..., 6
%d	Day of the month as a zero-padded decimal.	01, 02, ..., 31
%-d	Day of the month as a decimal number.	1, 2, ..., 30
%b	Abbreviated month name.	Jan, Feb, ..., Dec

Directive	Meaning	Example
%B	Full month name.	January, February, ...
%m	Month as a zero-padded decimal number.	01, 02, ..., 12
%-m	Month as a decimal number.	1, 2, ..., 12
%y	Year without century as a zero-padded decimal number.	00, 01, ..., 99
%-y	Year without century as a decimal number.	0, 1, ..., 99
%Y	Year with century as a decimal number.	2013, 2019 etc.
%H	Hour (24-hour clock) as a zero-padded decimal number.	00, 01, ..., 23
%-H	Hour (24-hour clock) as a decimal number.	0, 1, ..., 23
%I	Hour (12-hour clock) as a zero-padded decimal number.	01, 02, ..., 12
%-I	Hour (12-hour clock) as a decimal number.	1, 2, ... 12
%p	Locale’s AM or PM.	AM, PM
%M	Minute as a zero-padded decimal number.	00, 01, ..., 59
%-M	Minute as a decimal number.	0, 1, ..., 59
%S	Second as a zero-padded decimal number.	00, 01, ..., 59
%-S	Second as a decimal number.	0, 1, ..., 59
%f	Microsecond as a decimal number, zero-padded on the left.	000000 - 999999
%z	UTC offset in the form +HHMM or -HHMM.	
%Z	Time zone name.	
%j	Day of the year as a zero-padded decimal number.	001, 002, ..., 366

Directive	Meaning	Example
%-j	Day of the year as a decimal number.	1, 2, ..., 366
%U	Week number of the year (Sunday as the first day of the week). All days in a new year preceding the first Sunday are considered to be in week 0.	00, 01, ..., 53
%W	Numarul saptamanii in an (Luni este prima). All days in a new year preceding the first Monday are considered to be in week 0.	00, 01, ..., 53
%c	Locale's appropriate date and time representation.	Mon Sep 30 07:06:05 2013
%x	Locale's appropriate date representation.	09/30/13
%X	Locale's appropriate time representation.	07:06:05
%%	A literal '%' character.	%

```
newdt = datetime.strptime('20150120', '%Y%m%d')
print(newdt)
```

```
2015-01-20 00:00:00
```

Tipurile de tip datetime au propria lor aritmetica. Operatii de tip adunare, scadere se fac in functie de momentele implicate iar rezultatele contin si ele elemente disparate (un numar de ani, un numar de luni, etc)

```
dt_now = datetime.now()
delta = dt_now - dt
print(delta)
print(dir(delta))
```

```
221411 days, 13:40:37.146514
['__abs__', '__add__', '__bool__', '__class__', '__delattr__',
'__dir__', '__divmod__', '__doc__', '__eq__', '__floordiv__',
'__format__', '__ge__', '__getattribute__', '__getstate__', '__gt__',
'__hash__', '__init__', '__init_subclass__', '__le__', '__lt__',
'__mod__', '__mul__', '__ne__', '__neg__', '__new__', '__pos__',
'__radd__', '__rdivmod__', '__reduce__', '__reduce_ex__', '__repr__',
```

```
'__rfloordiv__', '__rmod__', '__rmul__', '__rsub__', '__rtruediv__',
'__setattr__', '__sizeof__', '__str__', '__sub__', '__subclasshook__',
'__truediv__', 'days', 'max', 'microseconds', 'min', 'resolution',
'seconds', 'total_seconds']
```

Tipuri de date multiple

Tip	Denumire
Secvențial	lista
Secvențial	tuplu
Mapare	dict
Set	set
Set	frozenset

Liste

Liste: Initializare

Listele sunt secvențe ordonate care pot stoca mai multe tipuri de obiete. În realitate, listele sunt liste de referinte care pot "trimite" către orice alt fel de obiect.

listele sunt inițializate cu ajutorul parantezelor drepte.

```
al = [1,2,3,4,5,6,7,8,9]

print (type(al))

print(dir(al))

<class 'list'>
['__add__', '__class__', '__class_getitem__', '__contains__',
'__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__',
'__format__', '__ge__', '__getattribute__', '__getitem__',
'__getstate__', '__gt__', '__hash__', '__iadd__', '__imul__',
'__init__', '__init_subclass__', '__iter__', '__le__', '__len__',
'__lt__', '__mul__', '__ne__', '__new__', '__reduce__',
'__reduce_ex__', '__repr__', '__reversed__', '__rmul__',
'__setattr__', '__setitem__', '__sizeof__', '__str__',
'__subclasshook__', 'append', 'clear', 'copy', 'count', 'extend',
'index', 'insert', 'pop', 'remove', 'reverse', 'sort']

cl = ['as', 'xc', 'dh', 'tj', 'lk', 'fg']

cl.reverse() #ca si sort, reverse se face in-place

print (cl)

cl.sort()  #sort functioneaza doar daca elementele listei sunt
uniforme
```

```
cl.append('HAM')

print (cl)
```

```
['fg', 'lk', 'tj', 'dh', 'xc', 'as']
['as', 'dh', 'fg', 'lk', 'tj', 'xc', 'HAM']
```

```
cdl = [1, 'xc', 13, 'tj', [1,2,3,4], 'fg']
print(cdl)
```

```
[1, 'xc', 13, 'tj', [1, 2, 3, 4], 'fg']
```

```
nl = [12,43,65,234,876,435,23,21,34,32]

print(nl)

nl.sort()

print(nl)
```

```
[12, 43, 65, 234, 876, 435, 23, 21, 34, 32]
[12, 21, 23, 32, 34, 43, 65, 234, 435, 876]
```

```
nl.append('HAM')

print(nl)
```

```
[12, 21, 23, 32, 34, 43, 65, 234, 435, 876, 'HAM']
```

```
nl.sort()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_29189/2620101995.py in <cell line: 0>()
----> 1 nl.sort()

TypeError: '<' not supported between instances of 'str' and 'int'
```

```
nl.pop()
```

```
{"type":"string"}
```

```
print(nl)
```

```
[12, 21, 23, 32, 34, 43, 65, 234, 435, 876]
```

metoda sort sorteaza "in place" - nu returneaza nimic

```
print(nl.sort())
```

```
None

nl[1] = 1231221
print(nl)

[12, 1231221, 23, 32, 34, 43, 65, 234, 435, 876]

my_string = "hello world"

my_list = list(my_string)

print(my_list)

my_list[0] = 'j'

print(my_list)

my_string = "".join(my_list)

print(my_string)

['h', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd']
['j', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd']
jello world
```

Slicing

```
sl = [1,2,3,4,5,6,7,8,9,10]

print(sl[3])
print(type(sl[3]))


print(sl[3:4])
print(type(sl[3:4]))

print(sl[3:])

print(sl[:5])
print(sl[:])

print(sl[::2])

print(sl[::-2])

4
<class 'int'>
[4]
<class 'list'>
[4, 5, 6, 7, 8, 9, 10]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
[1, 3, 5, 7, 9]
[10, 8, 6, 4, 2]
```

List comprehension

```
patrate = [numar ** 2 for numar in sl]

print(patrate)

[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Se pot adauga conditii

```
patrate = [numar ** 2 for numar in sl if numar > 4]

print(patrate)

[25, 36, 49, 64, 81, 100]
```

Pentru a utiliza if - else este necesara o expresie diferita

```
numbers = [1,2,3,4,5,6,7,8,9,10]

micmare = ["mic" if number < 7 else "mare" for number in numbers]

print(micmare)

['mic', 'mic', 'mic', 'mic', 'mic', 'mic', 'mare', 'mare', 'mare',
'mare']

micmare2 = ["mic" if number < 7 else "mare" for number in numbers if
number %2 ==0]

print(micmare2)

['mic', 'mic', 'mic', 'mare', 'mare']

matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

flattened = [item for row in matrix for item in row]

print(matrix)
print(flattened)

[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
[1, 2, 3, 4, 5, 6, 7, 8, 9]

matrix = [[item for item in range(5)] for row in range(3)  ]

print(matrix)
```

```
[[0, 1, 2, 3, 4], [0, 1, 2, 3, 4], [0, 1, 2, 3, 4]]
```

Pentru listele si operatiile simple, comprehension este mai rapida, nu pentru ca foloseste alt algoritm ci pentru ca evita operatii de citire, evaluare, cautarea metodelor pe obiecte... etc.

```
import timeit

def squares(size):
    result = []
    for number in range(size):
        result.append(number*number)
    return result

def squares_comprehension(size):
    return [number*number for number in range(size)]

print(timeit.timeit("squares(50)", "from __main__ import squares",
number = 1000000))
print(timeit.timeit("squares_comprehension(50)", "from __main__ import
squares_comprehension", number = 1000000))
```

```
2.4666334070025187
2.093962681003177
```

Copiere, atribuire

Atribuirea copiaza doar referinta. Se creaza o variabila de tip lista noua dar elementele din interior sunt aceleasi

```
a = [1, 2, 3]
b = a
b[0] = 99
print(a)
```

```
[99, 2, 3]
```

Metoda copy() a unei liste poate fi folosita pentru copierea superficiala

```
a = [1, 2, 3]
b = a.copy()
b[0] = 99
print(a)
```

```
[1, 2, 3]
```

Dar daca lista contine ca elemente alte liste, copierea superficiala va copia doar referintele acestora.

```
a = [[1,2], [3,4]]
b = a.copy()
b[0][0] = 99
print(a)
```

```
[[99, 2], [3, 4]]
```

Copierea produnda - deepcopy - face o copie completa

```
import copy

a = [[1,2], [3,4]]
b = copy.deepcopy(a)
b[0][0] = 99
print(a)
```

```
[[1, 2], [3, 4]]
```

Despachetare (unpacking)

```
mll = [1990, 12, 32]

[an, luna, zi] = mll

print(type(an))
print(type(luna))
print(type(zi))

print(an, luna, zi)
```

```
<class 'int'>
<class 'int'>
<class 'int'>
1990 12 32
```

```
mlal = [1990, 12, 32, 56]

[an, luna, zi] = mlal
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_29189/1240910896.py in <cell line: 0>()
      1 mlal = [1990, 12, 32, 56]
      2
----> 3 [an, luna, zi] = mlal

ValueError: too many values to unpack (expected 3)
```

```
[*a, b, c] = [12,23,34,456,23,21,3434,3223]

print(a)
print(b)
print(c)

[*d] = [12,23,34,456,23,21,3434,3223]

print(d)
```

```
[12, 23, 34, 456, 23, 21]
3434
3223
[12, 23, 34, 456, 23, 21, 3434, 3223]
```

Tupluri

Un tuplu este o structură de date care reprezintă un grup fix de valori, ordonate și imutabile.

Inițial, tuplul nu a fost gândit ca o „colecție pentru utilizator”, ci ca o structură internă a interpreterului, folosită pentru a manipula eficient grupuri mici de valori: argumentele unei funcții, perechi (cheie, valoare), rezultate multiple etc. Pentru aceste situații, Python avea nevoie de un obiect simplu, de dimensiune fixă și fără modificări, iar tuple-ul îndeplinește exact acest rol.

În loc să ascundă această structură, Python a ales să o expună direct în limbaj. Astfel, ceea ce este folosit intern pentru organizarea datelor devine și un instrument disponibil programatorului.

Caracteristica esențială rămâne imutabilitatea: după creare, valorile nu se pot schimba. Din acest motiv, tuple-ul este stabil și potrivit pentru reprezentarea unor grupuri fixe de date care trebuie tratate ca o singură unitate.

```
tpl = (12,23,54,34,54)

print(type(tpl))

print(dir(tpl))
```

```
<class 'tuple'>
['__add__', '__class__', '__class_getitem__', '__contains__',
'__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
'__getattribute__', '__getitem__', '__getnewargs__', '__getstate__',
'__gt__', '__hash__', '__init__', '__init_subclass__', '__iter__',
'__le__', '__len__', '__lt__', '__mul__', '__ne__', '__new__',
'__reduce__', '__reduce_ex__', '__repr__', '__rmul__', '__setattr__',
'__sizeof__', '__str__', '__subclasshook__', 'count', 'index']
```

```
ttpl = (12,32,34, 'ala', 'bala', 4.6, 'porto', 32, 'cala')

print(ttpl)
```

```
(12, 32, 34, 'ala', 'bala', 4.6, 'porto', 32, 'cala')
```

tuplul este definit de prezenta virgulei, nu de paranteza

```
t = 1,2,3
print(t)
print(type(t))
```

```
(1, 2, 3)
<class 'tuple'>
```

un caz aparte

```
t1 = (1)
t2 = (1,)
print(type(t1))
print(type(t2))
```

```
<class 'int'>
<class 'tuple'>
```

Chiar daca un tuplu este imutabil, acesta poate contine obiecte care pot fi modificate

```
t3 = (1, [2,3])
t3[1].append(4)
print(t3)
```

```
(1, [2, 3, 4])
```

Unpacking

```
tvr = (1,2,3,4,5,6)
a, b, *c = tvr

print(a, b, c)
```

```
1 2 [3, 4, 5, 6]
```

```
print (type(tvr))
print (type(a))
print (type(b))
print (type(c))
```

```
<class 'tuple'>
<class 'int'>
```

```
<class 'int'>
<class 'list'>

argumente = (12,43)

print(type(argumente))

msstr = "sirul meu contine si doua chestii din afara: %d, %d" %
argumente

print(msstr)

<class 'tuple'>
sirul meu contine si doua chestii din afara: 12, 43
```

tuplul este folosit implicit in transferul argumentelor catre si dinspre functii, in functii speciale (ex: items())

```
dictionar = {'a' : 1, 'b' : 2, 'c' : 3}
for k, v in dictionar.items():
    print(k, v)

for k in dictionar.items():
    print(type(k))


a 1
b 2
c 3
<class 'tuple'>
<class 'tuple'>
<class 'tuple'>
```

Seturi (multimi)

Un hash este o funcție care transformă un obiect într-un număr (de obicei un întreg), astfel încât acel obiect să poată fi localizat rapid într-o structură de date. Ideea este simplă: în loc să cauți un element parcurgând toate valorile (ca într-o listă), calculezi hash-ul lui și mergi direct în locul unde ar trebui să fie. Pentru ca acest mecanism să funcționeze corect, obiectele folosite trebuie să aibă un hash stabil, adică să nu se modifice în timp.

Pe baza acestui mecanism sunt construite set-urile din Python. Un set este o structură care stochează elemente folosind hash-ul lor, ceea ce permite verificări foarte rapide de tipul „există acest element?”. Din acest motiv, set-urile nu au ordine și nu permit duplicate: fiecare element este identificat prin hash și apare o singură dată. Astfel, set-ul nu este o listă mai limitată, ci o structură specializată pentru testarea rapidă a apartenenței și eliminarea automată a duplicatelor.

```
mset = {"mere", "pere", "prune"}

print(mset)
print(type(mset))
print(dir(mset))
```

```
{'mere', 'prune', 'pere'}
<class 'set'>
['__and__', '__class__', '__class_getitem__', '__contains__',
'__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
'__getattribute__', '__getstate__', '__gt__', '__hash__', '__iand__',
'__init__', '__init_subclass__', '__ior__', '__isub__', '__iter__',
'__ixor__', '__le__', '__len__', '__lt__', '__ne__', '__new__',
'__or__', '__rand__', '__reduce__', '__reduce_ex__', '__repr__',
'__ror__', '__rsub__', '__rxor__', '__setattr__', '__sizeof__',
'__str__', '__sub__', '__subclasshook__', '__xor__', 'add', 'clear',
'copy', 'difference', 'difference_update', 'discard', 'intersection',
'intersection_update', 'isdisjoint', 'issubset', 'issuperset', 'pop',
'remove', 'symmetric_difference', 'symmetric_difference_update',
'union', 'update']
```

```
mset = {"mere", "pere", "prune", "mere", "mere", "mere", "mere",
"mere", "mere", "mere", "mere", "mere" }

print(mset)
```

```
{'mere', 'prune', 'pere'}
```

```
mlist = ["mere", "pere", "prune", "mere", "mere", "mere", "mere",
"mere", "mere", "mere", "mere", "mere" ]

print(mlist)

unique = list(set(mlist))

print(unique)
```

```
['mere', 'pere', 'prune', 'mere', 'mere', 'mere', 'mere', 'mere',
'mere', 'mere', 'mere', 'mere']
['mere', 'prune', 'pere']
```

seturile nu pot fi indexate

```
print(mset[2])
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_29189/3255088463.py in <cell line: 0>()
----> 1 print(mset[2])
```

```
TypeError: 'set' object is not subscriptable
```

Putem transforma in lista

```
print(list(mset)[2])

pere
```

dar nu e corect pentru ca NU ne putem baza NICIODATA pe faptul ca setul isi va mentine ordinea de la adaugare

```
print({"apple", "banana", "cherry", "date", "aple"})

{'apple', 'aple', 'banana', 'date', 'cherry'}
```

setul nu poate contine obiecte mutabile pentru ca hash-ul acestora ar trebui fie sa se modifice odata cu modificarea lor. În principiu, ar fi posibil ca obiectele mutabile (cum sunt listele) să fie folosite pe post de chei și să li se refacă hash-ul atunci când se modifică. Însă asta ar necesita mecanisme suplimentare: obiectele ar trebui să notifice dicționarele că s-au schimbat, iar acestea ar trebui să-și reorganizeze intern toate pozițiile afectate. Rezultatul ar fi un sistem mai complex, mai lent și mai greu de înțeles. Din acest motiv, Python face o alegere de design clară: permite doar obiecte imutabile ca fiind hashabile, pentru a păstra performanța și comportamentul predictibil al structurilor bazate pe hashing.

```
{ [1,2] }   # error

---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_29189/1726537524.py in <cell line: 0>()
----> 1 { [1,2] }   # error

TypeError: unhashable type: 'list'
```

Tuplul, datorita imutabilitatii sale, poate face parte dintr-un set

```
set_t = { (1,2) }
```

Operatii tipice cu seturi

Uniune

```
set1 = {"fag", "frasin", "stejar", "alun"}
set2 = {"fag", "mesteacan", "balsa", "brad", "cais", "stejar"}

print(set1.union(set2))
```

```
{'brad', 'frasin', 'mesteacan', 'fag', 'alun', 'stejar', 'balsa',
'cais'}
```

Intersectie

```
set1 = {"fag", "frasin", "stejar", "alun"}
set2 = {"fag", "mesteacan", "balsa", "brad", "cais", "stejar"}

print(set1.intersection(set2))

{'stejar', 'fag'}
```

Diferenta

```
set1 = {"fag", "frasin", "stejar", "alun"}
set2 = {"fag", "mesteacan", "balsa", "brad", "cais", "stejar"}

print(set1.difference(set2))
print(set2.difference(set1))

{'frasin', 'alun'}
{'brad', 'mesteacan', 'balsa', 'cais'}
```

Diferenta simetrica

Diferenta simetrica dintre doua multimi retine toate elementele care nu fac parte din intersectia acestora

```
set1 = {"fag", "frasin", "stejar", "alun"}
set2 = {"fag", "mesteacan", "balsa", "brad", "cais", "stejar"}

print(set1.symmetric_difference(set2))
print(set2.symmetric_difference(set1))

{'brad', 'mesteacan', 'alun', 'frasin', 'balsa', 'cais'}
{'brad', 'frasin', 'mesteacan', 'alun', 'balsa', 'cais'}
```

issuperset

Este setul curent supersetul unui alt set?

```
set1 = {"fag", "frasin", "stejar", "alun", "brad", "cais", "stejar"}
set2 = {"brad", "cais", "stejar"}

print(set1.issuperset(set2))
print(set2.issuperset(set1))

True
False
```

difference_update, intersection_update si symmetric_difference_update

difference_update: Inlatura dintr-un set elementele altui set

intersection_update: Adauga la set (daca nu exista deja) intersectia dintre setul curent si un alt set

symmetric_difference_update: Adauga la set (daca nu exista deja) diferenta simetrica dintre setul curent si un alt set

```
set1 = {"fag", "frasin", "stejar", "alun", "brad", "cais", "stejar"}
set2 = {"brad", "cais", "stejar", "par"}

set1.difference_update(set2)
print(set1)
```

```
{'fag', 'alun', 'frasin'}
```

```
set1 = {"fag", "frasin", "stejar", "alun", "brad", "cais", "stejar"}
set2 = {"brad", "cais", "stejar", "par"}

set1.intersection_update(set2)
print(set1)
```

```
{'brad', 'stejar', 'cais'}
```

```
set1 = {"fag", "frasin", "stejar", "alun", "brad", "cais", "stejar"}
set2 = {"brad", "cais", "stejar", "par"}

set1.symmetric_difference_update(set2)
print(set1)
```

```
{'fag', 'alun', 'par', 'frasin'}
```